

Einundzwanzig Meetup App — UX/UI Upgrade Protokoll

Für Claude (Anthropic) — Schritt-für-Schritt Implementierung

Projekt-Kontext

App: Einundzwanzig Meetup App

Repo: <https://github.com/louisthecat86/Einundzwanzig-Meetup-App>

Framework: Flutter 3.41+ / Dart 3.10+

Design-System: Dark Theme, Bitcoin-Orange Akzent (#F7931A), Rajdhani Font

Ziel: Die App von einem soliden, funktionalen Design auf ein **premium, emotional ansprechendes Nutzererlebnis** heben.

Bestehende Stärken (nicht ändern):

- WCAG-konforme Kontraste
 - Konsistente Farbpalette in `theme.dart`
 - Glassmorphism-NavBar mit `BackdropFilter`
 - Reorderable Dashboard-Kacheln
 - Generative Art für Badge-Karten
 - Lokalisierung (de/en/es)
-

Inhaltsverzeichnis

1. Phase 1: Micro-Interactions & Feedback (Quick Wins)
 2. Phase 2: Dashboard-Visual-Upgrade
 3. Phase 3: Badge-Sammel-Erlebnis (Celebration)
 4. Phase 4: Reputation Card Redesign
 5. Phase 5: Scanner-Ansicht Premium
 6. Phase 6: Onboarding & Coach Marks
 7. Phase 7: Animationen & Motion
 8. Phase 8: Accessibility & Polish
 9. Phase 9: Badge Visual Identity — Meetup Cover & Event Badges
-

Phase 1: Micro-Interactions & Feedback (Quick Wins)

Geschätzter Aufwand: 3–4 Stunden

Impact: Sehr hoch

1.1 Scale-Down-Animation für alle tappable Elemente

Erstelle eine wiederverwendbare Widget-Komponente, die auf alle Kacheln, Buttons und Listeneinträge angewendet wird.

Neue Datei: `lib/widgets/pressable.dart`

```

import 'package:flutter/material.dart';

/// Ein Wrapper, der auf Tap-Down leicht skaliert und auf Tap-Up
/// wieder zurückfedert. Gibt dem Nutzer sofortiges haptisches Feedback.
class Pressable extends StatefulWidget {
  final Widget child;
  final VoidCallback? onTap;
  final VoidCallback? onLongPress;
  final double scaleDownTo;
  final Duration duration;

  const Pressable({
    super.key,
    required this.child,
    this.onTap,
    this.onLongPress,
    this.scaleDownTo = 0.97,
    this.duration = const Duration(milliseconds: 100),
  });

  @override
  State<Pressable> createState() => _PressableState();
}

class _PressableState extends State<Pressable>
  with SingleTickerProviderStateMixin {
  late final AnimationController _controller;
  late final Animation<double> _animation;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      vsync: this,
      duration: widget.duration,
      reverseDuration: const Duration(milliseconds: 80),
    );
    _animation = Tween<double>(begin: 1.0, end: widget.scaleDownTo)
      .animate(CurvedAnimation(
        parent: _controller,
        curve: Curves.easeOutCubic,
        reverseCurve: Curves.easeInCubic,
      ));
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }
}

```

```

void _onTapDown(TapDownDetails _) => _controller.forward();
void _onTapUp(TapUpDetails _) => _controller.reverse();
void _onTapCancel() => _controller.reverse();

@override
Widget build(BuildContext context) {
  return GestureDetector(
    onTapDown: _onTapDown,
    onTapUp: _onTapUp,
    onTapCancel: _onTapCancel,
    onTap: widget.onTap,
    onLongPress: widget.onLongPress,
    behavior: HitTestBehavior.translucent,
    child: AnimatedBuilder(
      animation: _animation,
      builder: (_, child) => Transform.scale(
        scale: _animation.value,
        child: child,
      ),
      child: widget.child,
    ),
  );
}
}

```

Anwendung in `_tile()` (`home_screen.dart`):

```

// In der _tile() Methode, um den gesamten Kachel-Content:
Widget _tile({required Widget child, required Color accentColor, VoidCallback? onTap, ...}) {
  return Pressable(
    onTap: onTap,
    child: Container(
      // ... bestehende Container-Dekoration
      child: ClipRRect(... child ...),
    ),
  );
}

```

Anwendung auf Buttons:

```

// Für ElevatedButton im Theme oder als Wrapper:
Pressable(
  onTap: onPressed,
  child: Container(
    // Button-Dekoration
    child: Center(child: Text('Label')),
  ),
)

```

1.2 Haptic Pattern System

Neue Datei: lib/services/haptic_service.dart

```
import 'package:flutter/services.dart';
import 'package:shared_preferences/shared_preferences.dart';

/// Zentralisiertes Haptic-Feedback mit konsistenten Patterns.
/// Prüft vor jedem Feedback, ob Haptic vom Nutzer aktiviert ist.
class HapticService {
  static Future<bool> _isEnabled() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getBool('haptic_enabled') ?? true;
  }

  static Future<void> light() async {
    if (!await _isEnabled()) return;
    HapticFeedback.selectionClick();
  }

  static Future<void> medium() async {
    if (!await _isEnabled()) return;
    HapticFeedback.mediumImpact();
  }

  static Future<void> heavy() async {
    if (!await _isEnabled()) return;
    HapticFeedback.heavyImpact();
  }

  static Future<void> success() async {
    if (!await _isEnabled()) return;
    HapticFeedback.notificationFeedback(type: NotificationFeedbackType.success);
  }

  static Future<void> error() async {
    if (!await _isEnabled()) return;
    HapticFeedback.notificationFeedback(type: NotificationFeedbackType.error);
  }
}
```

Integration: Ersetze alle `HapticFeedback.*` Aufrufe in der App durch `HapticService.*` Aufrufe.

1.3 Kachel-Tap-Overlay-Feedback

Erweitere `_tile()` in `home_screen.dart` um einen visuellen Tap-Overlay:

```
Widget _tile({required Widget child, required Color accentColor, VoidCallback? onTap, ...}) {
  return Pressable(
    onTap: onTap != null ? () async {
      await HapticService.light();
```

```

        onTap();
    } : null,
    child: Container(
      decoration: _buildingPinnedTile
        ? BoxDecoration(
            borderRadius: BorderRadius.circular(kTileRadius),
            gradient: LinearGradient(
              begin: Alignment.topLeft,
              end: Alignment.bottomRight,
              colors: [
                cOrange.withValues(alpha: 0.13),
                cOrange.withValues(alpha: 0.04),
                const Color(0xFF141416),
              ],
              stops: const [0.0, 0.45, 1.0],
            ),
            border: Border.all(color: cOrange.withValues(alpha: 0.30), width: 1.0),
            boxShadow: [
              BoxShadow(
                color: cOrange.withValues(alpha: 0.05),
                blurRadius: 20,
                offset: const Offset(0, 5),
              ),
            ],
          )
        : BoxDecoration(
            color: cCard,
            borderRadius: BorderRadius.circular(kTileRadius),
            border: Border.all(color: cTileBorder, width: 0.5),
          ),
      child: ClipRRect(
        borderRadius: BorderRadius.circular(kTileRadius),
        child: Stack(
          children: [
            // ... bestehende Watermarks etc.
            Positioned.fill(
              child: Padding(
                padding: const EdgeInsets.fromLTRB(16, 16, 16, 18),
                child: child,
              ),
            ),
          ],
        ),
      ),
    ),
  );
}

```

Phase 2: Dashboard-Visual-Upgrade

Geschätzter Aufwand: 6–8 Stunden

Impact: Hoch

2.1 Elevation-Aware Shadows

Neue Datei: lib/widgets/shadows.dart

```
import 'package:flutter/material.dart';
import '../theme.dart';

/// Erzeugt Schatten basierend auf der visuellen Elevation.
/// Kleine Elemente = enger, harter Schatten.
/// Große Elemente = weicher, diffuser Schatten.
List<BoxShadow> shadowForElevation(double elevation, {Color? accent}) {
  final color = accent ?? cOrange;

  if (elevation <= 0) return [];

  if (elevation <= 2) {
    // Subtil — für kleine Kacheln, Buttons
    return [
      BoxShadow(
        color: color.withValues(alpha: 0.06),
        blurRadius: 8,
        spreadRadius: -2,
        offset: const Offset(0, 2),
      ),
    ];
  }

  if (elevation <= 4) {
    // Medium — für Standard-Kacheln
    return [
      BoxShadow(
        color: color.withValues(alpha: 0.05),
        blurRadius: 16,
        spreadRadius: -4,
        offset: const Offset(0, 4),
      ),
    ];
  }

  // High — für Hero-Kacheln, Bottom Sheets
  return [
    BoxShadow(
      color: color.withValues(alpha: 0.08),
      blurRadius: 32,
      spreadRadius: -8,
      offset: const Offset(0, 8),
    ),
  ];
}
```

```

BoxShadow(
  color: Colors.black.withValues(alpha: 0.2),
  blurRadius: 16,
  spreadRadius: -4,
  offset: const Offset(0, 4),
),
];
}

```

Anwendung: Ersetze alle hartcodierten boxShadow Arrays durch shadowForElevation(4) etc.

2.2 Farbhierarchie für Kacheln (semantische Akzente)

Das Dashboard nutzt aktuell überall Orange. Führe semantische Akzentfarben ein:

Erweiterung in theme.dart:

```

// Semantische Akzentfarben für Dashboard-Kacheln
const Color cAccentPrimary   = Color(0xFFFF7931A); // Orange — Bitcoin, primär
const Color cAccentSuccess   = Color(0xFF22C55E); // Grün — Verifiziert, OK
const Color cAccentInfo      = Color(0xFF00B4D8); // Cyan — Info, Portal
const Color cAccentCommunity = Color(0xFFD4A017); // Gold — Community
const Color cAccentNostr     = Color(0xFF8B5CF6); // Lila — Nostr
const Color cAccentDanger    = Color(0xFFE53935); // Rot — Warnung
const Color cAccentNeutral   = Color(0xFF868A94); // Grau — Inaktiv

```

Mapping für Tile-Builder (in _initTileDefs):

```

// Jedem Tile eine semantische Farbe zuweisen:
// home_meetup      -> cAccentPrimary
// reputation       -> cAccentCommunity (Gold/Amber)
// trust_network    -> cAccentSuccess
// community        -> cAccentInfo
// events           -> cAccentPrimary
// converter        -> cAccentPrimary
// btc_dashboard    -> cAccentPrimary
// news             -> cAccentPrimary
// portal           -> cAccentInfo
// satoshiduell     -> Color(0xFFFFC93C) // Gold
// plebrap          -> cAccentPrimary
// nostr            -> cAccentNostr
// organisator      -> cAccentPrimary
// wot_dashboard    -> cAccentSuccess

```

Wichtig: Die home_meetup Kachel behält den goldenen Verlauf. Alle anderen Kacheln nutzen nur einen subtilen Rahmen in ihrer Akzentfarbe statt des vollen Orange-Gradients.

2.3 Dashboard-Kontextualität (zeitbasiert)

Das Dashboard sollte sich je nach Kontext anders verhalten:

Neue Datei: lib/services/dashboard_context.dart

```
import '../models/calendar_event.dart';

enum DashboardContext { normal, preMeetup, duringMeetup, postMeetup }

class DashboardContextService {
  /// Bestimmt den aktuellen Dashboard-Kontext basierend auf dem
  /// nächsten anstehenden Meetup.
  static DashboardContext determine(CalendarEvent? nextEvent) {
    if (nextEvent == null) return DashboardContext.normal;

    final now = DateTime.now();
    final diff = nextEvent.startTime.difference(now);

    if (diff.isNegative) {
      // Meetup läuft gerade (Startzeit in der Vergangenheit,
      // angenommen Meetups dauern ~3h)
      if (diff.inHours > -4) return DashboardContext.duringMeetup;
      return DashboardContext.postMeetup;
    }

    if (diff.inDays <= 3) return DashboardContext.preMeetup;
    return DashboardContext.normal;
  }

  static bool shouldHighlightMeetup(DashboardContext ctx) =>
    ctx == DashboardContext.preMeetup || ctx == DashboardContext.duringMeetup;
}
```

Integration in home_screen.dart:

```
// In build() oder initState:
DashboardContext _context = DashboardContext.normal;

void _updateContext() {
  final next = _favCards.isNotEmpty ? _favCards.first.event : null;
  setState(() => _context = DashboardContextService.determine(next));
}

// In _buildHomeMeetupTile():
// Wenn _context == duringMeetup, zeige einen pulsierenden
// "LIVE"-Badge auf der Home-Meetup-Kachel.
```

Phase 3: Badge-Sammel-Erlebnis (Celebration)

Geschätzter Aufwand: 4–6 Stunden

Impact: Sehr hoch

3.1 Konfetti-Animation beim Badge-Erfolg

Package hinzufügen: confetti: ^0.8.0

In `meetup_verification.dart` nach erfolgreichem Speichern:

```
import 'package:confetti/confetti.dart';

class MeetupVerificationScreen extends StatefulWidget {
  // ... bestehend
}

class _MeetupVerificationScreenState extends State<MeetupVerificationScreen>
  with SingleTickerProviderStateMixin {

  late ConfettiController _confettiController;

  @override
  void initState() {
    super.initState();
    _confettiController = ConfettiController(
      duration: const Duration(seconds: 3),
    );
    // ... restliches initState
  }

  @override
  void dispose() {
    _confettiController.dispose();
    // ... restliches dispose
  }

  Future _confirmSaveBadge() async {
    // ... bestehende Logik bis zum Speichern

    // NACH erfolgreichem Speichern:
    _confettiController.play();
    await HapticService.success();

    // Animated Score-Update im Header (wenn möglich)
    // ...

    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Row(children: [
            const Icon(Icons.celebration, color: Colors.white),
            const SizedBox(width: 10),
            Expanded(child: Text(AppLocalizations.of(context).verifyBadgeSaved)),
          ]),
          backgroundColor: cGreen,
          behavior: SnackBarBehavior.floating,
        ),
      );
    }
  }
}
```

```

        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
        duration: const Duration(seconds: 4),
      ),
    );
  }

  // ... Rest der Navigation
}

// In _buildConfirmationView():
Widget _buildConfirmationView() {
  return Stack(
    children: [
      // ... bestehende Confirmation-UI

      // Konfetti-Overlay
      Align(
        alignment: Alignment.topCenter,
        child: ConfettiWidget(
          confettiController: _confettiController,
          blastDirectionality: BlastDirectionality.explosive,
          particleDrag: 0.05,
          emissionFrequency: 0.05,
          numberOfParticles: 30,
          gravity: 0.2,
          colors: const [
            Color(0xFFFF7931A), // Bitcoin Orange
            Color(0xFFFFD700), // Gold
            Color(0xFFFFFFFF), // Weiß
            Color(0xFFFFB347), // Light Orange
          ],
        ),
      ),
    ],
  );
}
}

```

3.2 Hero-Animation: Badge “fliegt” in die Wallet

In `meetup_verification.dart`:

```

// Wenn der Nutzer "Zur Wallet hinzufügen" tippt:
Future _confirmSaveBadge() async {
  // ... nach Speichern:

  if (mounted) {
    // Hero-Navigation statt pushReplacement
    Navigator.pushReplacement(
      context,

```

```

PageRouteBuilder(
  pageBuilder: (_, animation, __) => const BadgeWalletScreen(),
  transitionsBuilder: (_, animation, __, child) {
    // Fade + leichte Scale-Up
    return FadeTransition(
      opacity: animation,
      child: ScaleTransition(
        scale: Tween<double>(begin: 0.95, end: 1.0).animate(
          CurvedAnimation(parent: animation, curve: Curves.easeOutCubic),
        ),
        child: child,
      ),
    );
  },
  transitionDuration: const Duration(milliseconds: 400),
);
}

```

3.3 Trust Score Puls-Animation beim Update

In `home_screen.dart` — Profile Header:

```

// Ersetze die statische Score-Plakette durch eine animierte Version:
Widget _buildProfileHeader() {
  return Row(children: [
    // ... Avatar

    // Trust Score mit Animation
    GestureDetector(
      onTap: () => Navigator.push(context, MaterialPageRoute(
        builder: (_) => const ReputationCardScreen(),
      )),
      child: AnimatedTrustScore(
        score: _trustScore?.totalScore ?? 0.0,
        level: _trustScore?.level ?? 'NEU',
      ),
    ),
  ]);
}

```

Neue Datei: `lib/widgets/animated_trust_score.dart`

```

import 'package:flutter/material.dart';
import '../theme.dart';

class AnimatedTrustScore extends StatefulWidget {
  final double score;
  final String level;
}

```

```

const AnimatedTrustScore({
  super.key,
  required this.score,
  required this.level,
});

@override
State<AnimatedTrustScore> createState() => _AnimatedTrustScoreState();
}

class _AnimatedTrustScoreState extends State<AnimatedTrustScore>
  with SingleTickerProviderStateMixin {
  late AnimationController _pulseController;
  double _displayedScore = 0.0;

  @override
  void initState() {
    super.initState();
    _pulseController = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 600),
    );
    _animateTo(widget.score);
  }

  @override
  void didUpdateWidget(covariant AnimatedTrustScore oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (oldWidget.score != widget.score) {
      _animateTo(widget.score);
    }
  }

  void _animateTo(double target) async {
    _pulseController.forward(from: 0.0);
    // Einfache Zähl-Animation
    final start = _displayedScore;
    final diff = target - start;
    const steps = 20;
    for (int i = 1; i <= steps; i++) {
      await Future.delayed(const Duration(milliseconds: 25));
      if (mounted) {
        setState(() => _displayedScore = start + (diff * i / steps));
      }
    }
  }

  @override
  void dispose() {
    _pulseController.dispose();
  }
}

```

```

    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return AnimatedBuilder(
      animation: _pulseController,
      builder: (_, child) {
        final scale = 1.0 + (_pulseController.value * 0.08);
        return Transform.scale(
          scale: scale,
          child: child,
        );
      },
      child: Container(
        padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 7),
        decoration: BoxDecoration(
          gradient: LinearGradient(
            begin: Alignment.topLeft,
            end: Alignment.bottomRight,
            colors: [
              cOrange,
              cOrange.withValues(alpha: 0.78),
            ],
          ),
          borderRadius: BorderRadius.circular(12),
        ),
        child: Row(mainAxisSize: MainAxisSize.min, children: [
          Text(
            _displayedScore.toStringAsFixed(1),
            style: const TextStyle(
              color: Colors.black,
              fontSize: 17,
              fontWeight: FontWeight.w900,
            ).copyWith(fontFamily: fontMono),
          ),
          const SizedBox(width: 6),
          const Icon(Icons.chevron_right_rounded, color: Colors.black54, size: 16),
        ]),
      ),
    );
  }
}

```

Phase 4: Reputation Card Redesign

Geschätzter Aufwand: 6–8 Stunden

Impact: Sehr hoch

4.1 Die Reputation Card als “Teilen-swertes” Asset

Neue Datei: lib/widgets/reputation_card_visual.dart

Ersetze den weißen QR-Code-Hintergrund durch eine designed Karte:

```
import 'package:flutter/material.dart';
import '../theme.dart';

/// Eine visuell ansprechende Reputation-Karte, die screenshot- und
/// teilenswert ist. Dunkler Hintergrund, Gold-Akzente, klare Hierarchie.
class ReputationCardVisual extends StatelessWidget {
  final String nickname;
  final double score;
  final String level;
  final int badgeCount;
  final int meetupCount;
  final int signerCount;
  final Widget qrCode;
  final bool isSigned;

  const ReputationCardVisual({
    super.key,
    required this.nickname,
    required this.score,
    required this.level,
    required this.badgeCount,
    required this.meetupCount,
    required this.signerCount,
    required this.qrCode,
    required this.isSigned,
  });

  Color get _levelColor {
    switch (level) {
      case 'VETERAN': return Colors.amber;
      case 'ETABLIERT': return Colors.green;
      case 'AKTIV': return cCyan;
      case 'STARTER': return cOrange;
      default: return cTextTertiary;
    }
  }

  IconData get _levelIcon {
    switch (level) {
      case 'VETERAN': return Icons.bolt;
      case 'ETABLIERT': return Icons.shield;
      case 'AKTIV': return Icons.local_fire_department;
      case 'STARTER': return Icons.eco;
      default: return Icons.fiber_new;
    }
  }
}
```

```

@override
Widget build(BuildContext context) {
  return Container(
    width: 340,
    padding: const EdgeInsets.all(24),
    decoration: BoxDecoration(
      gradient: LinearGradient(
        begin: Alignment.topLeft,
        end: Alignment.bottomRight,
        colors: [
          const Color(0xFF1A1A1D),
          const Color(0xFF0C0C0E),
        ],
      ),
    borderRadius: BorderRadius.circular(24),
    border: Border.all(
      color: _levelColor.withValues(alpha: 0.4),
      width: 1.5,
    ),
    boxShadow: [
      BoxShadow(
        color: _levelColor.withValues(alpha: 0.15),
        blurRadius: 30,
        spreadRadius: -5,
        offset: const Offset(0, 10),
      ),
    ],
  ),
  child: Column(
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      // --- HEADER: Level Icon + Name ---
      Row(
        children: [
          Container(
            width: 48,
            height: 48,
            decoration: BoxDecoration(
              color: _levelColor.withValues(alpha: 0.15),
              shape: BoxShape.circle,
              border: Border.all(
                color: _levelColor.withValues(alpha: 0.4),
                width: 1.5,
              ),
            ),
          ),
          Icon(_levelIcon, color: _levelColor, size: 24),
          const SizedBox(width: 14),
          Expanded(
            child: Column(

```

```

        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            nickname.toUpperCase(),
            style: const TextStyle(
              color: cText,
              fontSize: 16,
              fontWeight: FontWeight.w800,
              letterSpacing: 0.5,
            ),
            maxLines: 1,
            overflow: TextOverflow.ellipsis,
          ),
          const SizedBox(height: 2),
          Text(
            level,
            style: TextStyle(
              color: _levelColor,
              fontSize: 12,
              fontWeight: FontWeight.w700,
              letterSpacing: 1.2,
            ),
          ),
        ],
      ),
    ],
  ),
),

const SizedBox(height: 20),

// --- SCORE (groß) ---
Container(
  padding: const EdgeInsets.symmetric(vertical: 16, horizontal: 24),
  decoration: BoxDecoration(
    color: _levelColor.withValues(alpha: 0.08),
    borderRadius: BorderRadius.circular(16),
    border: Border.all(
      color: _levelColor.withValues(alpha: 0.2),
      width: 1,
    ),
  ),
),
child: Column(
  children: [
    Text(
      'TRUST SCORE',
      style: TextStyle(
        color: _levelColor.withValues(alpha: 0.7),
        fontSize: 10,
        fontWeight: FontWeight.w800,
        letterSpacing: 2,

```

```

    ),
  ),
  const SizedBox(height: 4),
  Text(
    score.toStringAsFixed(1),
    style: TextStyle(
      color: _levelColor,
      fontSize: 42,
      fontWeight: FontWeight.w900,
      height: 1.0,
    ).copyWith(fontFamily: fontMono),
  ),
  ],
),
),

const SizedBox(height: 20),

// --- STATS ROW ---
Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
  children: [
    _statColumn('$badgeCount', 'BADGES', cOrange),
    _divider(),
    _statColumn('$meetupCount', 'MEETUPS', cCyan),
    _divider(),
    _statColumn('$signerCount', 'SIGNER', cPurple),
  ],
),

const SizedBox(height: 24),
const Divider(color: cBorder, height: 1),
const SizedBox(height: 20),

// --- QR CODE ---
Container(
  padding: const EdgeInsets.all(12),
  decoration: BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.circular(16),
  ),
  child: qrCode,
),

const SizedBox(height: 12),

// --- SIGNIERT-STATUS ---
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Icon(

```

```

        isSigned ? Icons.verified_user : Icons.lock_outline,
        color: isSigned ? cGreen : cTextTertiary,
        size: 14,
      ),
      const SizedBox(width: 6),
      Text(
        isSigned ? 'SCHNORR SIGNIERT' : 'UNSIGNIERT',
        style: TextStyle(
          color: isSigned ? cGreen : cTextTertiary,
          fontSize: 10,
          fontWeight: FontWeight.w700,
          letterSpacing: 1.5,
        ),
      ),
    ],
  ),
),

const SizedBox(height: 8),

// --- FOOTER ---
Text(
  'EINUNDZWANZIG • PROOF OF ATTENDANCE',
  style: TextStyle(
    color: cTextTertiary.withValues(alpha: 0.6),
    fontSize: 9,
    fontWeight: FontWeight.w600,
    letterSpacing: 1.5,
  ),
),
],
),
);
}

Widget _statColumn(String value, String label, Color color) {
  return Column(
    children: [
      Text(
        value,
        style: TextStyle(
          color: color,
          fontSize: 18,
          fontWeight: FontWeight.w800,
        ),
      ),
      const SizedBox(height: 2),
      Text(
        label,
        style: const TextStyle(
          color: cTextTertiary,
          fontSize: 9,

```

```

        fontWeight: FontWeight.w700,
        letterSpacing: 1.2,
      ),
    ),
  ],
);
}

Widget _divider() {
  return Container(
    width: 1,
    height: 24,
    color: cBorder,
  );
}
}

```

Integration in reputation_qr.dart:

```

// Ersetze _buildQRCard() durch:
Widget _buildQRCard(BuildContext context) {
  return Center(
    child: RepaintBoundary(
      key: _qrRepaintKey,
      child: ReputationCardVisual(
        nickname: _user.nickname.isNotEmpty ? _user.nickname : 'Anon',
        score: _trustScore?.totalScore ?? 0.0,
        level: _trustScore?.level ?? 'NEU',
        badgeCount: _verifiedBadgeCount,
        meetupCount: _trustScore?.uniqueMeetups ?? 0,
        signerCount: _trustScore?.uniqueSigners ?? 0,
        qrCode: QrImageView(
          data: _qrData,
          version: QrVersions.auto,
          size: 180,
          backgroundColor: Colors.white,
          errorCorrectionLevel: QrErrorCorrectLevel.L,
        ),
        isSigned: _hasIdentity,
      ),
    ),
  );
}

```

Phase 5: Scanner-Ansicht Premium

Geschätzter Aufwand: 5–7 Stunden

Impact: Hoch

5.1 Radar-Wellen-Animation für NFC

Neue Datei: lib/widgets/radar_animation.dart

```
import 'package:flutter/material.dart';
import '../theme.dart';

/// Konzentrische Kreise, die ausgehen wie ein Radar.
/// Nutzt einen AnimationController, der von außen gesteuert wird.
class RadarAnimation extends StatelessWidget {
  final AnimationController controller;
  final int ringCount;
  final Color color;

  const RadarAnimation({
    super.key,
    required this.controller,
    this.ringCount = 3,
    this.color = cOrange,
  });

  @override
  Widget build(BuildContext context) {
    return AnimatedBuilder(
      animation: controller,
      builder: (_, __) {
        return Stack(
          alignment: Alignment.center,
          children: List.generate(ringCount, (index) {
            // Jeder Ring hat einen Offset, damit sie versetzt pulsieren
            final delay = index / ringCount;
            final t = ((controller.value + delay) % 1.0);

            final opacity = (1.0 - t) * 0.4;
            final scale = 0.5 + (t * 0.5);
            final strokeWidth = 2.0 * (1.0 - t);

            return Transform.scale(
              scale: scale,
              child: Container(
                width: 200,
                height: 200,
                decoration: BoxDecoration(
                  shape: BoxShape.circle,
                  border: Border.all(
                    color: color.withValues(alpha: opacity),
                    width: strokeWidth,
                  ),
                ),
              ),
            );
          }),
        );
      },
    );
  }
}
```

```

    );
  },
);
}
}

```

Integration in `meetup_verification.dart`:

```

// Ersetze den ScaleTransition-Kreis durch:
Widget _buildScannerView() {
  return Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      Stack(
        alignment: Alignment.center,
        children: [
          // Radar-Wellen
          RadarAnimation(controller: _controller, ringCount: 3),

          // Zentrales NFC-Icon
          Container(
            width: 120,
            height: 120,
            decoration: BoxDecoration(
              shape: BoxShape.circle,
              color: cOrange.withValues(alpha: 0.1),
              border: Border.all(color: cOrange, width: 2),
            ),
            child: const Icon(Icons.nfc, size: 48, color: Colors.white),
          ),
        ],
      ),
      // ... restliche UI
    ],
  );
}

```

5.2 QR-Scanner mit Overlay-Rahmen

In `_QRScannerScreen`:

```

class _QRScannerScreenState extends State<_QRScannerScreen> {
  bool _isScanned = false;
  bool _isDetecting = false; // Für Rahmen-Animation

  void _onDetect(BarcodeCapture capture) {
    if (_isScanned) return;
    for (final barcode in capture.barcodes) {
      final String? code = barcode.rawValue;
      if (code != null) {

```

```

try {
  final data = json.decode(code) as Map<String, dynamic>;
  if (data.containsKey('t') || data.containsKey('type')) {
    setState(() {
      _isScanned = true;
      _isDetecting = true;
    });
    // Kurze Erfolgs-Animation vor dem Pop
    Future.delayed(const Duration(milliseconds: 300), () {
      if (mounted) Navigator.pop(context, data);
    });
    return;
  }
} catch (_) {}
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: cDark,
    appBar: AppBar(title: Text(AppLocalizations.of(context).verifyScanQr)),
    body: Stack(
      children: [
        MobileScanner(onDetect: _onDetect),

        // Dunkles Overlay außerhalb des Scan-Bereichs
        CustomPaint(
          size: MediaQuery.of(context).size,
          painter: _ScannerOverlayPainter(
            cutoutSize: 250,
            borderColor: _isDetecting ? Colors.green : cOrange,
          ),
        ),

        // Anleitung
        Positioned(
          bottom: 60,
          left: 40,
          right: 40,
          child: AnimatedContainer(
            duration: const Duration(milliseconds: 200),
            padding: const EdgeInsets.all(16),
            decoration: BoxDecoration(
              color: _isDetecting
                ? Colors.green.withValues(alpha: 0.8)
                : Colors.black.withValues(alpha: 0.7),
              borderRadius: BorderRadius.circular(12),
            ),
            child: Text(

```

```

        _isDetecting
        ? '$$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
        ↪ emoji/2713.png}}}}}$\$ QR-Code erkannt!'
        : AppLocalizations.of(context).verifyScanQrInstruction,
        style: const TextStyle(color: Colors.white, fontSize: 14),
        textAlign: TextAlign.center,
    ),
),
),
],
),
);
}
}

class _ScannerOverlayPainter extends CustomPainter {
  final double cutoutSize;
  final Color borderColor;

  _ScannerOverlayPainter({
    required this.cutoutSize,
    required this.borderColor,
  });

  @override
  void paint(Canvas canvas, Size size) {
    final paint = Paint()
      ..color = Colors.black.withValues(alpha: 0.6)
      ..style = PaintingStyle.fill;

    final cutout = RRect.fromRectAndRadius(
      Rect.fromCenter(
        center: Offset(size.width / 2, size.height / 2 - 40),
        width: cutoutSize,
        height: cutoutSize,
      ),
      const Radius.circular(20),
    );

    // Dunkler Hintergrund mit Aussparung
    final path = Path()
      ..addRect(Rect.fromLTWH(0, 0, size.width, size.height))
      ..addRRect(cutout)
      ..fillType = PathFillType.evenOdd;

    canvas.drawPath(path, paint);

    // Ecken-Rahmen
    final borderPaint = Paint()
      ..color = borderColor
      ..style = PaintingStyle.stroke

```

```

        ..strokeWidth = 3;

final cornerLength = 30.0;
final rect = cutout.outerRect;

// Oben links
canvas.drawLine(
    rect.topLeft.translate(0, cornerLength),
    rect.topLeft,
    borderPaint,
);
canvas.drawLine(
    rect.topLeft.translate(cornerLength, 0),
    rect.topLeft,
    borderPaint,
);

// Oben rechts
canvas.drawLine(
    rect.topRight.translate(0, cornerLength),
    rect.topRight,
    borderPaint,
);
canvas.drawLine(
    rect.topRight.translate(-cornerLength, 0),
    rect.topRight,
    borderPaint,
);

// Unten links
canvas.drawLine(
    rect.bottomLeft.translate(0, -cornerLength),
    rect.bottomLeft,
    borderPaint,
);
canvas.drawLine(
    rect.bottomLeft.translate(cornerLength, 0),
    rect.bottomLeft,
    borderPaint,
);

// Unten rechts
canvas.drawLine(
    rect.bottomRight.translate(0, -cornerLength),
    rect.bottomRight,
    borderPaint,
);
canvas.drawLine(
    rect.bottomRight.translate(-cornerLength, 0),
    rect.bottomRight,
    borderPaint,

```

```

    );
}

@override
bool shouldRepaint(covariant _ScannerOverlayPainter oldDelegate) =>
    oldDelegate.borderColor != borderColor;
}

```

Phase 6: Onboarding & Coach Marks

Geschätzter Aufwand: 5–6 Stunden

Impact: Hoch

6.1 Tutorial Coach Marks

Package hinzufügen: tutorial_coach_mark: ^1.2.11

In home_screen.dart:

```

import 'package:tutorial_coach_mark/tutorial_coach_mark.dart';

class HomeScreenState extends State<HomeScreen>
    with TickerProviderStateMixin, WidgetsBindingObserver {

    // Keys für Coach Marks
    final GlobalKey _scanFabKey = GlobalKey();
    final GlobalKey _walletTabKey = GlobalKey();
    final GlobalKey _scoreHeaderKey = GlobalKey();

    TutorialCoachMark? _tutorialCoachMark;

    @override
    void initState() {
        super.initState();
        // ... bestehendes initState

        // Coach Mark nach kurzem Delay zeigen (nur beim ersten Start)
        _checkAndShowTutorial();
    }

    Future<void> _checkAndShowTutorial() async {
        final prefs = await SharedPreferences.getInstance();
        final shown = prefs.getBool('tutorial_shown_v1') ?? false;
        if (shown) return;

        // Warte, bis das Layout steht
        await Future.delayed(const Duration(milliseconds: 800));
        if (!mounted) return;

        _showTutorial();
    }
}

```

```

    await prefs.setBool('tutorial_shown_v1', true);
}

void _showTutorial() {
  final targets = [
    TargetFocus(
      identify: 'scan_fab',
      keyTarget: _scanFabKey,
      alignSkip: Alignment.bottomRight,
      enableOverlayTab: true,
      contents: [
        TargetContent(
          align: ContentAlign.top,
          child: Container(
            padding: const EdgeInsets.all(16),
            decoration: BoxDecoration(
              color: cCard,
              borderRadius: BorderRadius.circular(16),
              border: Border.all(color: cOrange),
            ),
          ),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              const Icon(Icons.qr_code_scanner, color: cOrange, size: 32),
              const SizedBox(height: 10),
              Text(
                'Hier scannst du dein erstes Badge',
                style: const TextStyle(
                  color: cText,
                  fontSize: 16,
                  fontWeight: FontWeight.w700,
                ),
                textAlign: TextAlign.center,
              ),
              const SizedBox(height: 6),
              Text(
                'Halte dein Telefon an einen NFC-Tag oder scanne einen QR-Code beim nächsten Meetup.',
                style: const TextStyle(color: cTextSecondary, fontSize: 13),
                textAlign: TextAlign.center,
              ),
            ],
          ),
        ),
        TargetFocus(
          identify: 'wallet_tab',
          keyTarget: _walletTabKey,
          alignSkip: Alignment.bottomRight,
          contents: [

```

```

TargetContent(
  align: ContentAlign.top,
  child: Container(
    padding: const EdgeInsets.all(16),
    decoration: BoxDecoration(
      color: cCard,
      borderRadius: BorderRadius.circular(16),
      border: Border.all(color: cOrange),
    ),
    child: const Column(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        Icon(Icons.style_rounded, color: cOrange, size: 32),
        SizedBox(height: 10),
        Text(
          'Deine Badge-Sammlung',
          style: TextStyle(
            color: cText,
            fontSize: 16,
            fontWeight: FontWeight.w700,
          ),
          textAlign: TextAlign.center,
        ),
        SizedBox(height: 6),
        Text(
          'Alle gesammelten Badges findest du hier. Jeder Besuch ist kryptographisch  

          ↳ beweisbar.',
          style: TextStyle(color: cTextSecondary, fontSize: 13),
          textAlign: TextAlign.center,
        ),
      ],
    ),
  ),
),
],
),
],
),
);

_tutorialCoachMark = TutorialCoachMark(
  targets: targets,
  colorShadow: Colors.black.withValues(alpha: 0.85),
  textSkip: 'Überspringen',
  textStyleSkip: const TextStyle(color: cTextSecondary, fontSize: 14),
  paddingFocus: 8,
  opacityShadow: 0.85,
  onFinish: () => _tutorialCoachMark?.finish(),
)..show(context: context);
}

// In build() — wickle den Scan-FAB in einen Container mit Key:
// Der FAB in app_shell.dart bekommt den Key übergeben oder

```

```
// wir fügen einen Wrapper hinzu.  
}
```

Wichtig: Die Keys müssen an die tatsächlichen Widgets in `app_shell.dart` durchgereicht werden. Alternativ kann das Tutorial auch komplett in `app_shell.dart` implementiert werden.

6.2 Empty States mit Illustrationen

Neue Datei: `lib/widgets/empty_state.dart`

```
import 'package:flutter/material.dart';  
import '../theme.dart';  
  
/// Emotional ansprechender Empty State mit Icon-Animation und CTA.  
class EmptyState extends StatelessWidget {  
  final IconData icon;  
  final String title;  
  final String subtitle;  
  final String? actionLabel;  
  final VoidCallback? onAction;  
  final Color accentColor;  
  
  const EmptyState({  
    super.key,  
    required this.icon,  
    required this.title,  
    required this.subtitle,  
    this.actionLabel,  
    this.onAction,  
    this.accentColor = cOrange,  
  });  
  
  @override  
  Widget build(BuildContext context) {  
    return Center(  
      child: Padding(  
        padding: const EdgeInsets.all(40),  
        child: Column(  
          mainAxisAlignment: MainAxisAlignment.center,  
          children: [  
            // Pulsierendes Icon  
            _PulsingIcon(icon: icon, color: accentColor),  
            const SizedBox(height: 24),  
            Text(  
              title,  
              style: const TextStyle(  
                color: cText,  
                fontSize: 20,  
                fontWeight: FontWeight.w800,  
                letterSpacing: 0.3,  

```

```

        ),
        textAlign: TextAlign.center,
    ),
    const SizedBox(height: 10),
    Text(
        subtitle,
        style: const TextStyle(
            color: cTextSecondary,
            fontSize: 14,
            height: 1.5,
        ),
        textAlign: TextAlign.center,
    ),
    if (actionLabel != null && onAction != null) ...[
        const SizedBox(height: 28),
        SizedBox(
            width: double.infinity,
            height: 48,
            child: ElevatedButton(
                onPressed: onAction,
                style: ElevatedButton.styleFrom(
                    backgroundColor: accentColor,
                    foregroundColor: Colors.black,
                    shape: RoundedRectangleBorder(
                        borderRadius: BorderRadius.circular(12),
                    ),
                ),
                child: Text(
                    actionLabel!,
                    style: const TextStyle(
                        fontWeight: FontWeight.w800,
                        fontSize: 14,
                    ),
                ),
            ),
        ),
    ],
],
),
),
);
}
}

```

```

class _PulsingIcon extends StatefulWidget {
    final IconData icon;
    final Color color;

    const _PulsingIcon({required this.icon, required this.color});

    @override

```

```

    State<_PulsingIcon> createState() => _PulsingIconState();
}

class _PulsingIconState extends State<_PulsingIcon>
    with SingleTickerProviderStateMixin {
    late AnimationController _controller;

    @override
    void initState() {
        super.initState();
        _controller = AnimationController(
            vsync: this,
            duration: const Duration(milliseconds: 2000),
        )..repeat();
    }

    @override
    void dispose() {
        _controller.dispose();
        super.dispose();
    }

    @override
    Widget build(BuildContext context) {
        return AnimatedBuilder(
            animation: _controller,
            builder: (_, child) {
                final scale = 1.0 + (0.1 * _controller.value);
                return Transform.scale(
                    scale: scale,
                    child: Container(
                        width: 100,
                        height: 100,
                        decoration: BoxDecoration(
                            color: widget.color.withValues(alpha: 0.1),
                            shape: BoxShape.circle,
                            border: Border.all(
                                color: widget.color.withValues(alpha: 0.3),
                                width: 1.5,
                            ),
                        ),
                        child: Icon(widget.icon, color: widget.color, size: 40),
                    ),
                );
            },
        );
    }
}

```

Anwendung in `badge_wallet.dart`:

```
// Ersetze _buildEmptyState() durch:
Widget _buildEmptyState(BuildContext context) {
  return EmptyState(
    icon: Icons.collections_bookmark_outlined,
    title: AppLocalizations.of(context).walletNoBadges,
    subtitle: AppLocalizations.of(context).walletNoBadgesSub,
    actionLabel: 'Meetup finden',
    onAction: () => Navigator.push(
      context,
      MaterialPageRoute(builder: (_) => const CalendarScreen()),
    ),
    accentColor: cOrange,
  );
}
```

Phase 7: Animationen & Motion

Geschätzter Aufwand: 8–10 Stunden

Impact: Mittel bis Hoch

7.1 Page Transitions (Shared Element)

Neue Datei: lib/widgets/page_transitions.dart

```
import 'package:flutter/material.dart';

/// Sammlung von Page-Transition-Helfern für konsistente Navigation.

/// Fade + Scale-Up — für modale Screens
Route<T> fadeScaleRoute<T>(Widget screen) {
  return PageRouteBuilder<T>{
    pageBuilder: (_, __, ___) => screen,
    transitionsBuilder: (_, animation, __, child) {
      return FadeTransition(
        opacity: animation,
        child: ScaleTransition(
          scale: Tween<double>(begin: 0.95, end: 1.0).animate(
            CurvedAnimation(parent: animation, curve: Curves.easeOutCubic),
          ),
          child: child,
        ),
      );
    },
    transitionDuration: const Duration(milliseconds: 300),
  );
}

/// Slide-Up — für Bottom-Sheet-ähnliche Screens
Route<T> slideUpRoute<T>(Widget screen) {
```

```

return PageRouteBuilder<T>{
  pageBuilder: (_, __, ___) => screen,
  transitionsBuilder: (_, animation, __, child) {
    return SlideTransition(
      position: Tween<Offset>(
        begin: const Offset(0, 0.15),
        end: Offset.zero,
      ).animate(CurvedAnimation(
        parent: animation,
        curve: Curves.easeOutCubic,
      )),
      child: FadeTransition(
        opacity: animation,
        child: child,
      ),
    );
  },
  transitionDuration: const Duration(milliseconds: 350),
};
}

/// Slide from Right — für hierarchische Navigation
Route<T> slideRightRoute<T>(Widget screen) {
  return PageRouteBuilder<T>{
    pageBuilder: (_, __, ___) => screen,
    transitionsBuilder: (_, animation, __, child) {
      return SlideTransition(
        position: Tween<Offset>(
          begin: const Offset(1, 0),
          end: Offset.zero,
        ).animate(CurvedAnimation(
          parent: animation,
          curve: Curves.easeOutCubic,
        )),
        child: child,
      );
    },
    transitionDuration: const Duration(milliseconds: 250),
  };
}

```

Anwendung: Ersetze alle `MaterialPageRoute` durch die passende Transition:

- Dashboard → Badge Details: `slideRightRoute()`
- Dashboard → Settings: `slideUpRoute()`
- Scan → Wallet: `fadeScaleRoute()`

7.2 Parallax-Scroll im Dashboard

In `home_screen.dart`:

```
// Ersetze den aktuellen Column-Build durch einen CustomScrollView
// mit Slivers für Parallax-Effekte:

@override
Widget build(BuildContext context) {
  final top = MediaQuery.of(context).padding.top;

  return CustomScrollView(
    slivers: [
      // Parallax-Header
      SliverToBoxAdapter(
        child: Padding(
          padding: EdgeInsets.fromLTRB(16, top + 12, 16, 0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              _buildLogoBar(),
              const SizedBox(height: 14),
              _buildProfileHeader(),
              const SizedBox(height: 18),
            ],
          ),
        ),
      ),
      // Sticky Home-Meetup (optional)
      SliverToBoxAdapter(
        child: Padding(
          padding: const EdgeInsets.symmetric(horizontal: 16),
          child: _buildHomeMeetupTile(),
        ),
      ),
      // Kachel-Block
      SliverToBoxAdapter(
        child: Padding(
          padding: const EdgeInsets.fromLTRB(16, kTileGap, 16, 12),
          child: _buildTileBlock(),
        ),
      ),
      // Verfügbare Sektion
      SliverToBoxAdapter(
        child: Padding(
          padding: const EdgeInsets.symmetric(horizontal: 16),
          child: _buildAvailableSection(),
        ),
      ),
    ],
  );
}
```

```

        // Bottom padding
        const SliverToBoxAdapter(
          child: SizedBox(height: 24),
        ),
      ],
    );
  }

```

Phase 8: Accessibility & Polish

Geschätzter Aufwand: 3–4 Stunden

Impact: Mittel (aber essenziell)

8.1 Reduced Motion Support

Neue Datei: lib/services/motion_preferences.dart

```

import 'package:flutter/material.dart';

/// Prüft, ob der Nutzer reduzierte Bewegung bevorzugt.
/// Alle Animationen sollten diesen Wert berücksichtigen.
class MotionPreferences {
  static bool reduceMotion(BuildContext context) {
    return MediaQuery.of(context).disableAnimations ||
      View.of(context).platformDispatcher.accessibilityFeatures
        .disableAnimations;
  }
}

```

Anwendung:

```

// In jedem AnimatedBuilder/AnimationController:
final reduceMotion = MotionPreferences.reduceMotion(context);

if (reduceMotion) {
  // Keine Animation, direkter Zustand
  return child;
}

// Sonst: normale Animation
return AnimatedBuilder(...);

```

8.2 Semantics für Screen Reader

Beispiel für Dashboard-Kacheln:

```

Widget _tile({required Widget child, required String semanticLabel, ...}) {
  return Pressable(

```

```

    onTap: onTap,
    child: Semantics(
      label: semanticLabel,
      button: true,
      enabled: onTap != null,
      child: Container(
        // ... Dekoration
        child: child,
      ),
    ),
  );
}

// Beim Aufruf:
_tile(
  semanticLabel: 'Bitcoin Dashboard. Aktueller Preis 85.000 Euro. Block 850.000. Doppeltippen zum
    ↩ Öffnen.',
  child: _buildBtcDashboardTileContent(),
)

```

8.3 AMOLED Black Mode

Erweiterung in **theme.dart**:

```

// Füge eine Theme-Variante hinzu:
final ThemeData appThemeAmoled = appTheme.copyWith(
  scaffoldBackgroundColor: Colors.black,
  colorScheme: appTheme.colorScheme.copyWith(
    surface: const Color(0xFF050505),
  ),
  cardTheme: appTheme.cardTheme.copyWith(
    color: const Color(0xFF0A0A0A),
  ),
);

// In main.dart oder einem Settings-Provider:
// Wenn der Nutzer "AMOLED Black" aktiviert hat:
// return MaterialApp(theme: useAmoled ? appThemeAmoled : appTheme);

```

Phase 9: Badge Visual Identity — Meetup Cover & Event Badges

Geschätzter Aufwand: 8–12 Stunden

Impact: Sehr hoch — unterscheidet die App fundamental von anderen Badge-Systemen

9.1 Meetup-Cover als Wasserzeichen auf Badges

Ziel: Jede Badge-Karte zeigt visuell, von welchem Meetup sie stammt. Das Meetup-Cover (aus dem CalendarEvent) wird als subtiler Hintergrund auf die Badge-Karte gelegt — nicht als dominantes Bild, sondern als atmosphärisches Wasserzeichen.

Neue Datei: lib/widgets/badge_card_with_cover.dart

```
import 'package:flutter/material.dart';
import '../theme.dart';

/// Eine Badge-Karte mit Meetup-Cover als Wasserzeichen-Hintergrund.
/// Das Cover wird stark gedimmt (Opacity ~0.08-0.12) und mit einem
/// Farb-Overlay versehen, damit der Text lesbar bleibt.
///
/// Die Karte besteht aus drei Ebenen:
/// 1. Meetup-Cover (Background, gedimmt, verschwommen)
/// 2. Farb-Overlay (Gradient, dunkel -> halbtransparent)
/// 3. Inhalt (Generative Art, Text, Metadaten)
class BadgeCardWithCover extends StatelessWidget {
  final Widget generativeArt;
  final String meetupName;
  final String? meetupCity;
  final String? coverImageUrl;
  final DateTime badgeDate;
  final bool isEventBadge;
  final Color? accentColor;

  const BadgeCardWithCover({
    super.key,
    required this.generativeArt,
    required this.meetupName,
    this.meetupCity,
    this.coverImageUrl,
    required this.badgeDate,
    this.isEventBadge = false,
    this.accentColor,
  });

  @override
  Widget build(BuildContext context) {
    final Color effectiveAccent = accentColor ?? (isEventBadge ? Colors.amber : cOrange);

    return Container(
      decoration: BoxDecoration(
        borderRadius: BorderRadius.circular(20),
        border: Border.all(
          color: effectiveAccent.withValues(alpha: isEventBadge ? 0.5 : 0.25),
          width: isEventBadge ? 1.5 : 1.0,
        ),
      ),
      boxShadow: shadowForElevation(isEventBadge ? 6 : 3, accent: effectiveAccent),
    ),
```

```

child: ClipRRect(
  borderRadius: BorderRadius.circular(20),
  child: Stack(
    children: [
      // EBENE 1: Meetup-Cover als Wasserzeichen
      if (coverImageUrl != null && coverImageUrl!.isNotEmpty)
        Positioned.fill(
          child: Opacity(
            opacity: 0.10,
            child: Image.network(
              coverImageUrl!,
              fit: BoxFit.cover,
              errorBuilder: (_, __, ___) => const SizedBox.shrink(),
              loadingBuilder: (_, child, progress) {
                if (progress == null) return child;
                return Container(color: cCard);
              },
            ),
          ),
        ),
      // EBENE 2: Farb-Overlay fuer Lesbarkeit
      Positioned.fill(
        child: Container(
          decoration: BoxDecoration(
            gradient: LinearGradient(
              begin: Alignment.topCenter,
              end: Alignment.bottomCenter,
              colors: [
                cCard.withValues(alpha: 0.3),
                cCard.withValues(alpha: 0.85),
                cCard.withValues(alpha: 1.0),
              ],
              stops: const [0.0, 0.5, 1.0],
            ),
          ),
        ),
      ),
      // EBENE 3: Inhalt
      Padding(
        padding: const EdgeInsets.fromLTRB(16, 16, 16, 18),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            // Generative Art (oben)
            Expanded(
              flex: 3,
              child: Center(child: generativeArt),
            ),
          ],
        ),
      ),
    ],
  ),
),

```

```

// Meetup-Name (subtil, als Kontext)
Row(
  children: [
    Icon(
      isEventBadge ? Icons.event_available : Icons.place_outlined,
      color: effectiveAccent.withValues(alpha: 0.7),
      size: 12,
    ),
    const SizedBox(width: 4),
    Expanded(
      child: Text(
        meetupCity ?? meetupName,
        style: TextStyle(
          color: effectiveAccent.withValues(alpha: 0.7),
          fontSize: 10,
          fontWeight: FontWeight.w700,
          letterSpacing: 0.8,
        ),
        maxLines: 1,
        overflow: TextOverflow.ellipsis,
      ),
    ),
  ],
),
const SizedBox(height: 6),

// Datum
Text(
  _formatDate(badgeDate),
  style: const TextStyle(
    color: cTextSecondary,
    fontSize: 11,
    fontWeight: FontWeight.w600,
  ),
),
],
),
),

// EVENT BADGE BADGE
if (isEventBadge)
  Positioned(
    top: 10,
    right: 10,
    child: Container(
      padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 3),
      decoration: BoxDecoration(
        color: Colors.amber.withValues(alpha: 0.15),
        borderRadius: BorderRadius.circular(6),
        border: Border.all(
          color: Colors.amber.withValues(alpha: 0.4),

```

```

        width: 0.5,
      ),
    ),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        Icon(
          Icons.star,
          color: Colors.amber.withValues(alpha: 0.9),
          size: 10,
        ),
        const SizedBox(width: 3),
        Text(
          'EVENT',
          style: TextStyle(
            color: Colors.amber.withValues(alpha: 0.9),
            fontSize: 8,
            fontWeight: FontWeight.w900,
            letterSpacing: 1.0,
          ),
        ),
      ],
    ),
  ],
),
),
],
);
}

String _formatDate(DateTime date) {
  return '${date.day.toString().padLeft(2, '0')}.${date.month.toString().padLeft(2,
    ↪ '0')}.${date.year}';
}
}

```

Integration in badge_wallet.dart:

```

// Ersetze den bestehenden Card-Builder in _buildBadgeCard():
Widget _buildBadgeCard(BuildContext context, BadgeModel badge) {
  return Pressable(
    onTap: () => Navigator.push(
      context,
      fadeScaleRoute(BadgeDetailsScreen(badge: badge)),
    ),
    child: BadgeCardWithCover(
      generativeArt: GenerativeBadgeArt(seed: badge.badgeHash),
      meetupName: badge.meetupName,
      meetupCity: badge.meetupCity,
      coverImageUrl: badge.meetupCoverUrl, // Muss im BadgeModel hinzugefuegt werden
    ),
  );
}

```

```

        badgeDate: badge.timestamp,
        isEventBadge: badge.badgeType == BadgeType.event,
        accentColor: badge.badgeType == BadgeType.event ? Colors.amber : null,
    ),
);
}

```

Erweiterung des **BadgeModel**:

```

// In models/badge.dart oder aehnlich:
enum BadgeType { meetup, event }

class BadgeModel {
    // ... bestehende Felder

    final BadgeType badgeType;
    final String? meetupCoverUrl;    // URL zum Meetup-Cover
    final String? meetupCity;        // Stadt-Name fuer schnelle Anzeige
    final String? eventId;           // Verknuepfung zum Kalender-Event

    BadgeModel({
        // ... bestehende Parameter
        this.badgeType = BadgeType.meetup,
        this.meetupCoverUrl,
        this.meetupCity,
        this.eventId,
    });

    // fromJson / toJson erweitern
}

```

Caching-Strategie fuer Covers:

```

// In services/image_cache_service.dart
// Nutze das bestehende cached_network_image Package (falls vorhanden)
// oder implementiere einen einfachen Memory-Cache:

class CoverImageCache {
    static final Map<String, ImageProvider> _cache = {};

    static ImageProvider get(String url) {
        if (_cache.containsKey(url)) return _cache[url]!;
        final provider = NetworkImage(url);
        _cache[url] = provider;
        return provider;
    }

    static void clear() => _cache.clear();
}

```

9.2 Spezialbadges fuer Kalender-Events

Ziel: Organisatoren von Events (die im Kalender eingetragen sind) koennen Spezialbadges fuer ihre Events erstellen. Teilnehmer erhalten diese Badges durch denselben Mechanismus wie Meetup-Badges: NFC/QR-Scan + Standortabgleich + gleichzeitiges Vor-Ort-Sein.

Neue Datei: lib/screens/create_event_badge.dart

```
import 'package:flutter/material.dart';
import '../theme.dart';
import '../models/calendar_event.dart';
import '../services/badge_service.dart';
import '../services/location_service.dart';

/// Screen fuer Organisatoren, um ein Event-Badge zu erstellen.
/// Nur sichtbar/erreichbar, wenn der Nutzer Organisator eines
/// aktuellen oder bevorstehenden Events ist.
class CreateEventBadgeScreen extends StatefulWidget {
  final CalendarEvent event;

  const CreateEventBadgeScreen({super.key, required this.event});

  @override
  State<CreateEventBadgeScreen> createState() => _CreateEventBadgeScreenState();
}

class _CreateEventBadgeScreenState extends State<CreateEventBadgeScreen> {
  bool _isAtLocation = false;
  bool _isCheckingLocation = true;
  bool _isCreating = false;
  String? _error;

  @override
  void initState() {
    super.initState();
    _checkLocation();
  }

  Future<void> _checkLocation() async {
    setState(() => _isCheckingLocation = true);
    try {
      // Standortabgleich: Ist der Organisator am Event-Ort?
      final position = await LocationService.getCurrentPosition();
      final distance = LocationService.calculateDistance(
        position.latitude,
        position.longitude,
        widget.event.latitude,
        widget.event.longitude,
      );

      // Toleranz: 500m um den Event-Ort
      setState(() => _isAtLocation = distance <= 500);
    } catch (e) {
      _error = e.toString();
    }
  }
}
```

```

    } catch (e) {
        setState(() => _error = 'Standort konnte nicht ermittelt werden');
    } finally {
        setState(() => _isCheckingLocation = false);
    }
}

Future<void> _createEventBadge() async {
    if (!_isAtLocation) {
        ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(
                content: Text('Du musst am Event-Ort sein, um ein Badge zu erstellen.'),
                backgroundColor: Colors.red,
            ),
        );
        return;
    }

    setState(() => _isCreating = true);
    try {
        final badge = await BadgeService.createEventBadge(
            eventId: widget.event.id,
            eventName: widget.event.name,
            eventDate: widget.event.startTime,
            location: widget.event.location,
            coverUrl: widget.event.coverImageUrl,
            organizerPubkey: widget.currentUserPubkey, // Aus dem User-Provider
        );

        if (mounted) {
            await HapticService.success();

            ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(
                    content: Text('Event-Badge "${widget.event.name}" erstellt!'),
                    backgroundColor: cGreen,
                ),
            );

            Navigator.pop(context, badge);
        }
    } catch (e) {
        if (mounted) {
            ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(
                    content: Text('Fehler: $e'),
                    backgroundColor: Colors.red,
                ),
            );
        }
    } finally {

```

```

        if (mounted) setState(() => _isCreating = false);
    }
}

@override
Widget build(BuildContext context) {
    final loc = AppLocalizations.of(context);

    return Scaffold(
        backgroundColor: cDark,
        appBar: AppBar(
            title: const Text('Event-Badge erstellen'),
            backgroundColor: Colors.transparent,
            elevation: 0,
        ),
        body: SingleChildScrollView(
            padding: const EdgeInsets.all(20),
            child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                    // Event-Preview Card
                    Container(
                        width: double.infinity,
                        padding: const EdgeInsets.all(20),
                        decoration: BoxDecoration(
                            gradient: LinearGradient(
                                begin: Alignment.topLeft,
                                end: Alignment.bottomRight,
                                colors: [
                                    Colors.amber.withValues(alpha: 0.15),
                                    cCard,
                                ],
                            ),
                        borderRadius: BorderRadius.circular(20),
                        border: Border.all(
                            color: Colors.amber.withValues(alpha: 0.3),
                            width: 1,
                        ),
                    ),
                    child: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: [
                            Row(
                                children: [
                                    Container(
                                        padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
                                        decoration: BoxDecoration(
                                            color: Colors.amber.withValues(alpha: 0.15),
                                            borderRadius: BorderRadius.circular(8),
                                        ),
                                    child: Row(

```

```

        mainAxisSize: MainAxisSize.min,
        children: [
          Icon(Icons.event, color: Colors.amber, size: 14),
          const SizedBox(width: 4),
          Text(
            'EVENT',
            style: TextStyle(
              color: Colors.amber,
              fontSize: 10,
              fontWeight: FontWeight.w900,
              letterSpacing: 1,
            ),
          ),
        ],
      ),
    ],
  ),
  const SizedBox(height: 14),
  Text(
    widget.event.name,
    style: const TextStyle(
      color: cText,
      fontSize: 22,
      fontWeight: FontWeight.w800,
      height: 1.2,
    ),
  ),
),
const SizedBox(height: 8),
Row(
  children: [
    const Icon(Icons.calendar_today, color: cTextSecondary, size: 14),
    const SizedBox(width: 6),
    Text(
      _formatDate(widget.event.startTime),
      style: const TextStyle(color: cTextSecondary, fontSize: 13),
    ),
  ],
),
const SizedBox(height: 4),
Row(
  children: [
    const Icon(Icons.place, color: cTextSecondary, size: 14),
    const SizedBox(width: 6),
    Expanded(
      child: Text(
        widget.event.location,
        style: const TextStyle(color: cTextSecondary, fontSize: 13),
        maxLines: 1,
        overflow: TextOverflow.ellipsis,
      ),
    ),
  ],
),

```

```

        ),
        1,
    ),
    1,
),
),
),

const SizedBox(height: 24),

// Standort-Status
Container(
  width: double.infinity,
  padding: const EdgeInsets.all(16),
  decoration: BoxDecoration(
    color: _isAtLocation
      ? cGreen.withValues(alpha: 0.1)
      : cOrange.withValues(alpha: 0.1),
    borderRadius: BorderRadius.circular(16),
    border: Border.all(
      color: _isAtLocation
        ? cGreen.withValues(alpha: 0.3)
        : cOrange.withValues(alpha: 0.3),
      width: 1,
    ),
  ),
  child: Row(
    children: [
      Icon(
        _isAtLocation ? Icons.location_on : Icons.location_off,
        color: _isAtLocation ? cGreen : cOrange,
        size: 24,
      ),
      const SizedBox(width: 12),
      Expanded(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              _isAtLocation
                ? 'Standort bestaetigt'
                : 'Standort-Pruefung',
              style: TextStyle(
                color: _isAtLocation ? cGreen : cOrange,
                fontSize: 14,
                fontWeight: FontWeight.w700,
              ),
            ),
            const SizedBox(height: 2),
            Text(
              _isAtLocation
                ? 'Du befindest dich am Event-Ort. Badge-Erstellung moeglich.'

```

```

        : _isCheckingLocation
        ? 'Pruefe deinen Standort...'
        : 'Du musst am Event-Ort sein, um ein Badge zu erstellen.',
      style: const TextStyle(
        color: cTextSecondary,
        fontSize: 12,
        height: 1.4,
      ),
    ),
  ],
),
],
),
),
],
),
),

const SizedBox(height: 24),

// Erklaerung
Text(
  'So funktioniert es',
  style: const TextStyle(
    color: cText,
    fontSize: 16,
    fontWeight: FontWeight.w800,
  ),
),
const SizedBox(height: 12),
_inforow('1', 'Badge wird mit Event-Daten erstellt'),
_inforow('2', 'Teilnehmer scannen NFC/QR am Event'),
_inforow('3', 'Standortabgleich prueft Anwesenheit'),
_inforow('4', 'Badge wird kryptographisch signiert'),

const SizedBox(height: 32),

// CTA
SizedBox(
  width: double.infinity,
  height: 52,
  child: ElevatedButton.icon(
    onPressed: (_isAtLocation && !_isCreating) ? _createEventBadge : null,
    icon: _isCreating
      ? const SizedBox(
        width: 18,
        height: 18,
        child: CircularProgressIndicator(
          strokeWidth: 2,
          color: Colors.black,
        ),
      )
      : const Icon(Icons.add_circle_outline),
  ),
),

```

```

        label: Text(
          _isCreating
            ? 'Wird erstellt...'
            : 'Event-Badge erstellen',
          style: const TextStyle(
            fontWeight: FontWeight.w800,
            fontSize: 15,
          ),
        ),
        style: ElevatedButton.styleFrom(
          backgroundColor: Colors.amber,
          foregroundColor: Colors.black,
          disabledBackgroundColor: Colors.amber.withValues(alpha: 0.3),
          disabledForegroundColor: Colors.black.withValues(alpha: 0.5),
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(14),
          ),
        ),
      ),
    ),
  ),
  const SizedBox(height: 12),

  Center(
    child: Text(
      'Das Badge ist sofort fuer alle Teilnehmer verfuegbar.',
      style: TextStyle(
        color: cTextTertiary.withValues(alpha: 0.6),
        fontSize: 11,
      ),
      textAlign: TextAlign.center,
    ),
  ),
],
),
),
);
}

Widget _infoRow(String number, String text) {
  return Padding(
    padding: const EdgeInsets.only(bottom: 10),
    child: Row(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Container(
          width: 24,
          height: 24,
          decoration: BoxDecoration(
            color: cOrange.withValues(alpha: 0.15),
            shape: BoxShape.circle,

```

```

    ),
    child: Center(
      child: Text(
        number,
        style: const TextStyle(
          color: cOrange,
          fontSize: 11,
          fontWeight: FontWeight.w800,
        ),
      ),
    ),
  ),
),
const SizedBox(width: 10),
Expanded(
  child: Text(
    text,
    style: const TextStyle(
      color: cTextSecondary,
      fontSize: 13,
      height: 1.5,
    ),
  ),
),
],
),
);
}

String _formatDate(DateTime date) {
  return '${date.day.toString().padLeft(2, '0')}.${date.month.toString().padLeft(2,
    ↪ '0')}.${date.year}, ${date.hour.toString().padLeft(2, '0')}:${date.minute.toString().padLeft(2,
    ↪ '0')} Uhr';
}
}

```

9.3 Event-Badge im Kalender anzeigen

In `calendar_screen.dart` oder `event_details.dart`:

```

// Füge jedem Event im Kalender einen "Badge erstellen"-Button hinzu,
// wenn der aktuelle Nutzer Organisator ist:

Widget _buildEventCard(CalendarEvent event) {
  final bool isOrganizer = event.organizerPubkeys.contains(_currentUserPubkey);
  final bool hasBadge = event.hasAssociatedBadge;
  final bool isUpcoming = event.startTime.isAfter(DateTime.now().subtract(const Duration(hours: 6)));

  return Container(
    // ... bestehende Event-Card-Dekoration
    child: Column(

```

```

crossAxisAlignment: CrossAxisAlignment.start,
children: [
  // ... bestehende Event-Info

  // Badge-Status / Erstellen-Button
  if (isOrganizer && isUpcoming)
    Padding(
      padding: const EdgeInsets.only(top: 12),
      child: hasBadge
        ? Container(
            padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
            decoration: BoxDecoration(
              color: Colors.amber.withValues(alpha: 0.1),
              borderRadius: BorderRadius.circular(8),
              border: Border.all(
                color: Colors.amber.withValues(alpha: 0.3),
                width: 0.5,
              ),
            ),
            child: Row(
              mainAxisAlignment: MainAxisAlignment.min,
              children: [
                Icon(Icons.verified, color: Colors.amber, size: 14),
                const SizedBox(width: 6),
                Text(
                  'Event-Badge aktiv',
                  style: TextStyle(
                    color: Colors.amber,
                    fontSize: 11,
                    fontWeight: FontWeight.w700,
                  ),
                ),
              ],
            ),
          )
        : Pressable(
            onTap: () => Navigator.push(
              context,
              slideUpRoute(CreateEventBadgeScreen(event: event)),
            ),
            child: Container(
              padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 8),
              decoration: BoxDecoration(
                gradient: LinearGradient(
                  colors: [
                    Colors.amber.withValues(alpha: 0.2),
                    Colors.amber.withValues(alpha: 0.1),
                  ],
                ),
                borderRadius: BorderRadius.circular(10),
                border: Border.all(

```

```

        color: Colors.amber.withValues(alpha: 0.4),
        width: 1,
      ),
    ),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        Icon(Icons.add_circle, color: Colors.amber, size: 16),
        const SizedBox(width: 6),
        Text(
          'Event-Badge erstellen',
          style: TextStyle(
            color: Colors.amber,
            fontSize: 12,
            fontWeight: FontWeight.w800,
          ),
        ),
      ],
    ),
  ),
),
],
),
);
}

```

9.4 Event-Badge Scanner-Logik

In `meetup_verification.dart` oder einem neuen `event_verification.dart`:

```

/// Erweitert die bestehende Badge-Speicher-Logik um Event-Badges.
/// Event-Badges werden genauso gespeichert wie Meetup-Badges,
/// haben aber einen anderen Typ und visuelle Kennzeichnung.

class EventBadgeVerificationService {
  /// Prueft, ob ein Event-Badge fuer das aktuelle Event existiert
  /// und ob der Nutzer am richtigen Ort ist.
  static Future<EventBadge?> findActiveEventBadge(Position userPosition) async {
    // 1. Hole alle aktiven Event-Badges aus der API/DB
    final activeEvents = await CalendarService.getActiveEvents();

    for (final event in activeEvents) {
      if (event.associatedBadgeId == null) continue;

      // 2. Standortabgleich
      final distance = LocationService.calculateDistance(
        userPosition.latitude,
        userPosition.longitude,
        event.latitude,

```

```

        event.longitude,
    );

    // 3. Zeitfenster-Pruefung (Event laeuft gerade oder beginnt bald)
    final now = DateTime.now();
    final isInTimeWindow = now.isAfter(
        event.startTime.subtract(const Duration(hours: 1)),
    ) && now.isBefore(
        event.endTime ?? event.startTime.add(const Duration(hours: 12)),
    );

    if (distance <= 500 && isInTimeWindow) {
        return EventBadge(
            badgeId: event.associatedBadgeId!,
            eventName: event.name,
            eventId: event.id,
            coverUrl: event.coverImageUrl,
        );
    }
}

return null;
}
}

```

9.5 Visuelle Unterscheidung in der Wallet

In `badge_wallet.dart` — Filter & Gruppierung:

```

// Fuege Filter-Chips ueber der Badge-Grid hinzu:
Widget _buildFilterChips() {
    final filters = [
        FilterChipData('Alle', null),
        FilterChipData('Meetups', BadgeType.meetup),
        FilterChipData('Events', BadgeType.event),
    ];

    return SingleChildScrollView(
        scrollDirection: Axis.horizontal,
        padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
        child: Row(
            children: filters.map((f) {
                final isSelected = _activeFilter == f.type;
                return Padding(
                    padding: const EdgeInsets.only(right: 8),
                    child: Pressable(
                        onTap: () => setState(() => _activeFilter = f.type),
                        child: Container(
                            padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 8),
                            decoration: BoxDecoration(

```

```

        color: isSelected ? cOrange.withValues(alpha: 0.15) : cCard,
        borderRadius: BorderRadius.circular(10),
        border: Border.all(
          color: isSelected ? cOrange.withValues(alpha: 0.4) : cBorder,
          width: 1,
        ),
      ),
      child: Text(
        f.label,
        style: TextStyle(
          color: isSelected ? cOrange : cTextSecondary,
          fontSize: 12,
          fontWeight: FontWeight.w700,
        ),
      ),
    ),
  ),
);
}).toList(),
),
);
}

// Gruppieren Badges nach Jahr und Typ:
// 2026 Events (z.B. Blocktrainer, Zitadelle)
// 2026 Meetups (Muenchen, Berlin, ...)

```

9.6 Badge-Detail-Ansicht fuer Events

In `badge_details.dart`:

```

// Wenn das Badge ein Event-Badge ist, zeige zusaetzliche Event-Infos:
Widget _buildEventBadgeDetails(BadgeModel badge) {
  if (badge.badgeType != BadgeType.event) return const SizedBox.shrink();

  return Container(
    margin: const EdgeInsets.only(top: 16),
    padding: const EdgeInsets.all(16),
    decoration: BoxDecoration(
      gradient: LinearGradient(
        colors: [
          Colors.amber.withValues(alpha: 0.1),
          Colors.amber.withValues(alpha: 0.02),
        ],
      ),
    ),
    borderRadius: BorderRadius.circular(16),
    border: Border.all(
      color: Colors.amber.withValues(alpha: 0.2),
      width: 1,
    ),
  ),

```

```

    ),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Row(
          children: [
            Icon(Icons.event, color: Colors.amber, size: 18),
            const SizedBox(width: 8),
            Text(
              'EVENT-BADGE',
              style: TextStyle(
                color: Colors.amber,
                fontSize: 11,
                fontWeight: FontWeight.w900,
                letterSpacing: 1.5,
              ),
            ),
          ],
        ),
        const SizedBox(height: 10),
        if (badge.eventId != null)
          Pressable(
            onTap: () => Navigator.push(
              context,
              slidePageRoute(EventDetailsScreen(eventId: badge.eventId!)),
            ),
            child: Row(
              children: [
                Expanded(
                  child: Text(
                    'Zum Kalender-Event',
                    style: TextStyle(
                      color: Colors.amber.withValues(alpha: 0.8),
                      fontSize: 13,
                      fontWeight: FontWeight.w700,
                    ),
                  ),
                ),
                Icon(
                  Icons.arrow_forward_ios,
                  color: Colors.amber.withValues(alpha: 0.6),
                  size: 14,
                ),
              ],
            ),
          ),
      ],
    ),
  );
}

```

9.7 Datenbank-/API-Erweiterungen (Backend-Hinweis)

Fuer Claude — falls das Backend ebenfalls angepasst werden muss:

```
// API-Schema-Erweiterungen (Konzept):  
//  
// POST /api/v1/event-badges  
// {  
//   "event_id": "uuid",  
//   "organizer_pubkey": "hex",  
//   "signature": "schnorr_sig",  
//   "location_proof": {  
//     "lat": 48.1351,  
//     "lng": 11.5820,  
//     "timestamp": 1234567890  
//   }  
// }  
//  
// GET /api/v1/event-badges/{event_id}  
// -> Gibt Badge-Metadaten zurueck  
//  
// GET /api/v1/badges?type=event  
// -> Filtert nur Event-Badges
```

Neue Packages (pubspec.yaml)

Füge folgende Dependencies hinzu:

```
dependencies:  
  # Bestehende Packages bleiben erhalten  
  
  # NEU: Animation & Interaktion  
  confetti: ^0.8.0  
  tutorial_coach_mark: ^1.2.11  
  auto_size_text: ^3.0.0  
  
  # NEU: QR-Scanner Overlay (falls nicht bereits vorhanden)  
  # mobile_scanner ist bereits im Projekt
```

Empfohlene Reihenfolge für Claude

1. **Phase 1.1 + 1.2** — Pressable Widget + HapticService (2h)
2. **Phase 2.1 + 2.2** — Shadow-System + semantische Farben (2h)
3. **Phase 3.1** — Konfetti + Badge Celebration (2h)
4. **Phase 4** — Reputation Card Redesign (3h)
5. **Phase 5** — Scanner-Visual-Upgrade (3h)

6. **Phase 6** — Coach Marks + Empty States (3h)
7. **Phase 7** — Page Transitions (2h)
8. **Phase 9.1** — Meetup-Cover als Wasserzeichen auf Badges (3h)
9. **Phase 9.2 + 9.3** — Event-Badge Erstellung + Kalender-Integration (4h)
10. **Phase 9.4 + 9.5 + 9.6** — Scanner-Logik + Wallet-Filter + Detail-Ansicht (3h)
11. **Phase 8** — Accessibility + AMOLED (2h)

Gesamtaufwand: ~29–33 Stunden verteilt auf 11 Phasen.

? Anweisungen für Claude

Claude, implementiere die obenstehenden Phasen Schritt für Schritt.

Wichtige Regeln:

1. Halte dich an das bestehende Design-System in `theme.dart` (Farben, Radien, Fonts).
2. Füge keine neuen großen Abhängigkeiten ohne Rücksprache hinzu.
3. Jede Phase sollte eigenständig funktionieren — committe nach jeder Phase.
4. Teste auf einem echten Gerät (Android/iOS), nicht nur im Emulator.
5. Behalte die bestehende Lokalisierung bei — neue Strings müssen in `app_localizations` aufgenommen werden.
6. Keine bestehende Funktionalität zerstören — die kryptographische Sicherheit hat Priorität.
7. **Event-Badges:** Die Standortabgleich-Logik muss denselben Sicherheitsstandard wie Meetup-Badges haben (PoP, Schnorr-Signatur).
8. **Meetup-Cover:** Falls kein Cover verfügbar ist, falle auf die bestehende Generative Art zurück — nie einen leeren/broken Hintergrund zeigen.

Beginne mit **Phase 1** und arbeite dich vor.